Context engineering and RAG

Lecture 2 · LLMOps with Databricks

In this lecture, we build the retrieval foundation for the arXiv Curator. The application discovers papers, parses their PDFs, prepares searchable chunks, and retrieves evidence that a language model can use when drafting a post. The same preparation pipeline feeds two serving options: Databricks AI Search and Lakebase Postgres.

We recommend Lakebase for the course project because it gives us control over SQL retrieval and supports scale to zero for idle compute. We also implement AI Search so that you can compare managed embeddings, self-managed embeddings, filtering, hybrid search, and reranking.

What you should be able to do

- Explain what belongs in a model's context and why more context can reduce answer quality.
- Inspect parsed documents and choose meaningful chunk boundaries that fit an embedding model's input window.
- Build an AI Search index or synchronize a table of embedded chunks into Lakebase.
- Compare semantic, filtered, and hybrid retrieval using repeatable queries.

Reading map

Pages	Topic
2	Context engineering and the RAG flow
3–6	Ingestion, parsing, chunking, and embeddings
7–9	Choosing a serving option and querying AI Search
10–12	Lakebase synced tables, native SQL, and the ai_search function
13–15	Synchronization, retrieval quality, and the practical assignment

The reference application is deliberately more elaborate than a basic paper-summary workflow needs to be. Abstracts may be enough for a simple post generator; full PDFs let us retrieve methods, results, and details absent from an abstract. Use the architecture to learn the operating patterns, then keep only the pieces your own application needs.

Lecture 3 continues with MCP, tool calling, and agent architecture. Here, our focus is preparing and serving the information those tools will retrieve.

What the model sees

Prompt engineering improves the instructions for a model call. Context engineering also decides which information to include, how to order it, what to compress, and what to leave out. That decision is repeated as an agent receives tool results or new user input.

Context component	Examples
Instructions	System prompt, task constraints, output requirements, and examples
User request	The current question, clarification, and preferences
Retrieved knowledge	Document chunks and their provenance
Tools and results	Available tool definitions and the observations they return
Conversation and state	Relevant earlier messages, summaries, memory, and current task state

All of these compete for a finite token budget. Reserve room for the model's response as well as the input. More retrieved text can increase latency, cost, contradictions, and distraction. The target is enough reliable evidence to answer the question, rather than the largest possible prompt.

Context, harness, and loop

Layer	Responsibility
Context engineering	What does the model see on this call? Select evidence, trim history, and include useful state.
Harness engineering	How does a run execute? Manage tools, permissions, approvals, validation, and the runtime.
Loop engineering	How does the agent continue or stop? Handle retries, verification, progress, and stopping conditions.

The RAG flow

User question → retrieve candidates → select evidence → assemble context → generate an answer

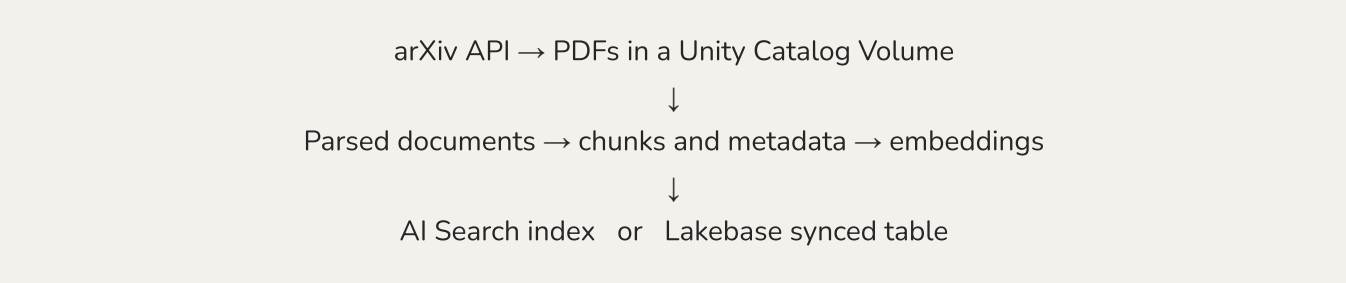
For vector retrieval, embed the query in the same vector space as the documents. Apply relevant metadata constraints, rank candidates, and include source identifiers with the selected chunks. Ask the generation model to ground its answer in that evidence and cite its sources.

Notebook 2.1 demonstrates query rewriting, context ordering, compression, and prompt assembly. These are separate interventions: generating alternate search queries can broaden retrieval; summarizing retrieved text reduces the context sent to the final model. Evaluate both. Compression can omit a qualification or number that matters to the answer.

Read more Anthropic context engineering

The shared data pipeline

The serving system changes; document preparation stays the same. `DataProcessor` in `src/arxiv_curator/data_processor.py` owns the ingestion, parsing, cleanup, and embedding steps.



Load the environment configuration

```
from pyspark.sql import SparkSession
from arxiv_curator.config import get_env, load_config
from arxiv_curator.data_processor import DataProcessor

spark = SparkSession.builder.getOrCreate()
cfg = load_config("../project_config.yml", get_env(spark))
processor = DataProcessor(spark=spark, config=cfg)
processor.process_and_save()
```

The configuration provides the catalog, schema, volume, embedding endpoint, Lakebase project, and usage policy. Use the configured environment rather than copying a developer's resource names into a notebook. The course currently uses `databricks-gte-large-en` for embeddings. The lecture moved away from the custom gateway embedding service because the demonstrated batch and search paths did not support it consistently.

What ingestion stores

Resource	Purpose
Unity Catalog Volume	PDF files grouped under the processing timestamp
`arxiv_papers`	Paper metadata: arxiv_id, title, authors, summary, pdf_url, published, processed, volume_path, ingest_ts
`ai_parsed_docs_table`	PDF path, parsed_content, and processing timestamp
`arxiv_chunks_table`	One row per text element, enriched with metadata and an embedding

The arXiv query selects `cs.AI` papers within a submission-date range. On the first run, the code looks back three days. Later runs use the maximum stored processing timestamp as a watermark. That reduces repeated work, but it is not a complete exactly-once ingestion design: a production workflow should also account for retries, duplicates, late discovery, and source updates.

Notebook 2.2 runs the processing entry point. A small corpus is enough for the course exercises; inspect it before expanding the ingestion volume and its associated cost.

Parse PDFs into usable content

PDFs preserve a visual layout, not a clean sequence of machine-readable paragraphs. Multi-column text, equations, captions, tables, headers, and scanned pages can all break reading order. Inspect the original PDF alongside the extracted content, especially for scientific papers.

The course uses `ai_parse_document` on binary PDF content read from a Volume. The processing table deliberately stores `parsed_content` as `STRING`, and the Python extraction logic reads it with `json.loads`.

```
# Shape of the course storage table
spark.sql(f"""
CREATE TABLE IF NOT EXISTS
{cfg.catalog}.{cfg.schema}.ai_parsed_docs_table (
    path STRING,
    parsed_content STRING,
    processed LONG
)
""")
```

Stored type versus function return type

The current function reference documents a `VARIANT` return value. That is structured, semi-structured data, rather than a plain text column. The course's persisted column is still a JSON string. Check the actual schema at the boundary where you consume it; a string in the table does not establish the native function's return type.

For an explicit conversion from the current function to JSON text, use:

```
SELECT path,
       to_json(ai_parse_document(content)) AS parsed_content
FROM READ_FILES('/Volumes/catalog/schema/volume/run',
                format => 'binaryFile');
```

Parsed elements are a starting point

The parser identifies content such as text, titles, section headers, tables, figures, and captions. The course extractor keeps only elements with `type == "text"`. This is a deliberate simplification: tables and figures may contain the strongest evidence in a paper, so excluding them can reduce retrieval quality.

There are two preparation paths. `ai_prep_search` supplies an opinionated approach to contextual chunks. The course instead parses the JSON and controls extraction and cleanup in Python. A parser's elements are not automatically the final chunking strategy for every use case.

Document Parsing also has a UI and REST API. Parsing is billable; page count and document complexity affect cost. The price examples in the lecture are planning estimates, not the total cost of ingestion, embeddings, storage, and retrieval.

Read more Parser and return type · Preparation function · Document Parsing API · Pricing

Prepare and inspect the chunks

A chunk is the unit retrieved for a question. It should carry a coherent piece of information and enough provenance to connect it back to its document. The course extractor walks `document.elements`, selects text, repairs hyphenation across line breaks, collapses whitespace, and creates one row per element. It does not remove stop words.

The resulting Delta table

Column	Meaning
`id`	Combined paper and chunk identifier; synchronization key
`arxiv_id`, `chunk_id`	Source paper identifier and element identifier
`text`	Cleaned chunk text
`title`, `summary`, `authors`	Metadata joined from arxiv_papers
`year`, `month`, `day`	Integer publication-date fields for filtering
`embedding`	ARRAY<FLOAT> generated from text using the configured model

`process_chunks()` generates the embedding with `ai_query`, appends the result to `arxiv_chunks_table`, and enables Delta Change Data Feed. Notebook 2.4 backfills missing vectors for older rows. A managed AI Search index can embed `text` itself; Lakebase and a self-managed AI Search index require the stored vectors.

Choose boundaries deliberately

Strategy	Trade-off
Fixed size with overlap	Predictable size and simple implementation. Overlap protects boundary context but duplicates content.
Sentences or paragraphs	More readable boundaries; enforce a maximum because lengths vary.
Structural or semantic	Follow sections and changes of topic. Preserve context, but add processing complexity.

Smaller chunks can improve precision but lose needed context. Larger chunks preserve more of a discussion but may mix topics and introduce noise. A recursive strategy can start with sections, then subsections, paragraphs, and smaller pieces where necessary.

Before creating the index

Sample different papers and compare chunks with their source. Inspect short fragments, repeated headers, missing captions, split tables, broken sentences, duplicate IDs, and missing metadata. Plot length distributions and count empty or oversized rows. Automated checks help at scale, but first inspect examples yourself to define what good looks like.

Notebook 2.3 is the inspection playground. Character lengths are useful for exploration; token counts from the selected embedding model's tokenizer are needed to enforce its input limit.

Embeddings and token windows

An embedding maps content to a vector of numbers. Similarity in that space can help find related meaning even when documents use different words. The vectors in this course represent text; other models can represent images, audio, or combinations of modalities.

Two sizes that must not be confused

Vector dimensions are the number of values in the output array. **The token window** is the maximum input the model can process. A 1,024-dimensional vector does not mean that the model accepts 1,024 input tokens.

Model in the lecture	Dimensions	Input window
databricks-gte-large-en	1,024	8,192 tokens
databricks-bge-large-en	1,024	512 tokens
text-embedding-3-small	1,536 by default	8,192 tokens
text-embedding-3-large	3,072 by default	8,192 tokens

For BGE, the complete embedding input must fit within 512 tokens. Leave space for any added title, prefix, or other metadata. A 512-character chunk is not the same thing as a 512-token chunk. The course's GTE configuration uses 1,024 dimensions, and the Lakebase type override must agree with that output.

Example

`MLOps is awesome` contains 15 characters including spaces. A tokenizer may split `MLOps` into subwords, so the phrase is not guaranteed to be three tokens. Token counts depend on the model's tokenizer. The rough "four characters per token" estimate is useful for planning, not for enforcing a hard limit.

Choosing and changing the model

Choose for language, domain, input type, retrieval quality, throughput, latency, and cost. A larger vector or input window does not guarantee better retrieval. The GTE and BGE models shown here are English-focused; multilingual workloads need evaluation in the languages that actually occur in the application.

The transcript's model-selection discussion emphasizes starting with a simple baseline and measuring improvement. Evaluate representative query-to-document relevance pairs, and also check the effect on the final application. A small retrieval improvement may not change answer quality enough to justify extra latency or cost.

Keep the embedding space consistent. Use the same model and compatible configuration for document and query embeddings. Changing the model means rebuilding document vectors and the corresponding index. Do not mix incompatible vectors merely because they have the same dimension.

HNSW means Hierarchical Navigable Small World, a graph-based method for approximate nearest-neighbor search. It is one possible index algorithm, not a synonym for embeddings or a fixed contract for every managed search service.

Read more Databricks embedding models · OpenAI embedding models

AI Search or Lakebase

Both routes start with the same Delta table of document chunks. The main decision is how much of serving and retrieval you want the platform to manage, and how much SQL-level control your application needs.

Decision	Databricks AI Search	Lakebase
Serving resource	AI Search endpoint and index	Postgres compute and a synced table
Document embeddings	Managed from text or supplied vectors	Generated before synchronization
Query embeddings	Automatic for managed indexes; application supplies vectors otherwise	Application embeds for direct SQL; ai_search can use the configured endpoint
Retrieval	Semantic search, hybrid search, metadata filtering, and reranking	Semantic search, hybrid search, and metadata filtering in SQL; custom reranking in the application
Operations	Managed index serving and synchronization	Managed database and synchronization, with indexes and queries you configure
Extensibility	Features exposed by the service	Supported Postgres extensions and SQL capabilities

The course recommendation

Use Lakebase for this project, particularly in development and acceptance environments with idle periods. With scale to zero enabled, idle compute can suspend. This does not eliminate storage, synchronization, embedding, or active-query costs. Connections and recurring work can also affect whether the compute becomes idle.

AI Search remains useful when you prefer its managed retrieval APIs and embedding workflow. Do not assume an idle serving endpoint is free just because no queries arrive. Compare the actual endpoint configuration, serving charges, indexing work, and embedding cost for your workload.

The lecture's discussion of price and scaling is a reason to investigate costs, not a universal performance benchmark. Avoid turning an approximate monthly example into a fixed price promise. Capacity, endpoint options, availability, and limits can change.

Use comparisons as a shortlist

The Superlinked comparison is useful for identifying features to investigate. Absence from a comparison table is not proof that a service cannot support a feature. Test the capabilities your application depends on, and read the platform's current documentation.

For Lakebase, distinguish open-source pgvector from the managed Lakebase service and its additional search extension. The supported extension catalog includes tools for text matching, geospatial queries, and diagnostics; installing arbitrary Postgres extensions is not implied.

Read more AI Search overview · Lakebase scale to zero · Supported extensions · Vector database comparison

Create the AI Search indexes

Notebook 2.4 creates a standard AI Search endpoint and demonstrates both embedding choices. The course uses `VectorSearchClient` from its existing SDK dependency; newer documentation also uses `AISearchClient`. Keep examples aligned with the installed package.

```
from databricks.vector_search.client import VectorSearchClient

ai_search = VectorSearchClient()
endpoint = f"{get_env(spark)}-arxiv-ai-search"
source = f"{cfg.catalog}.{cfg.schema}.arxiv_chunks_table"

if not ai_search.endpoint_exists(name=endpoint):
    ai_search.create_endpoint_and_wait(
        name=endpoint, endpoint_type="STANDARD",
        usage_policy_id=cfg.usage_policy_id,
    )
```

Two specifications for the same source

```
managed_name = f"{cfg.catalog}.{cfg.schema}.arxiv_chunks_ai_search_managed"
self_name = f"{cfg.catalog}.{cfg.schema}.arxiv_chunks_ai_search_self_managed"

specs = {
    managed_name: {
        "embedding_source_column": "text",
        "embedding_model_endpoint_name": cfg.embedding_endpoint,
    },
    self_name: {
        "embedding_vector_column": "embedding",
        "embedding_dimension": 1024,
    },
}
for name, spec in specs.items():
    if not ai_search.index_exists(endpoint_name=endpoint, index_name=name):
        ai_search.create_delta_sync_index(
            endpoint_name=endpoint, source_table_name=source,
            index_name=name, pipeline_type="TRIGGERED",
            primary_key="id", usage_policy_id=cfg.usage_policy_id,
            **spec,
        )
    else:
        ai_search.get_index(index_name=name).sync()
```

Managed embeddings use the text column and model endpoint to create document vectors. At query time, the index can accept text and embed it. The self-managed index reads `embedding` from Delta, so your application must generate the corresponding query vector.

Validate the primary key and source data before creation. The course enables Change Data Feed for incremental synchronization. A newly created index performs its first sync; a triggered index needs another sync after later source changes.

Wait for readiness before querying. Reusing an index is not the same as waiting for its latest update to finish: notebook 2.4 calls `wait_until_ready(wait_for_updates=True)` on both indexes.

Query AI Search

Keep the query, returned columns, metadata constraints, and ranking choices explicit. The examples below use the standard endpoint from notebook 2.4, whose filters are represented as a dictionary.

```
managed = ai_search.get_index(index_name=managed_name)
managed.wait_until_ready(wait_for_updates=True)
query = "transformer architecture attention mechanism"
columns = ["id", "arxiv_id", "title", "year", "text"]

results = managed.similarity_search(
    query_text=query,
    columns=columns,
    filters={"year": 2026},
    query_type="HYBRID",
    num_results=5,
)
```

`ANN` retrieves by vector similarity. `HYBRID` also uses keyword matching, which can help with acronyms, paper names, and specialized terms. Compare both on the same questions. Endpoint types differ in filter syntax; do not copy a SQL filter string from another configuration without checking it.

Self-managed embeddings

```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
client = w.serving_endpoints.get_open_ai_client()
vector = client.embeddings.create(
    model=cfg.embedding_endpoint, input=query
).data[0].embedding

self_managed = ai_search.get_index(index_name=self_name)
results = self_managed.similarity_search(
    query_vector=vector, columns=columns, num_results=5,
)
```

For a self-managed hybrid query, provide the query text for keyword matching as well as the query vector. The application owns embedding consistency; the index does not infer which model generated arbitrary vectors.

Filtering, thresholds, and reranking

A question such as “papers from 2026 about attention” contains both a topic and a date constraint. The application can place the topic in the query and the year in a metadata filter. The year comes from the prepared table, not from guessing dates in the chunk text.

A score threshold can return fewer results, including none; it should be calibrated against representative queries. Reranking evaluates retrieved candidates again, for example using `DatabricksReranker(columns_to_rerank=["text", "title", "summary"])`. It adds processing, so measure quality and latency together. Custom reranking can also run in application code after retrieval.

Read more Query API

Synchronize the chunks into Lakebase

Notebook 2.5 mirrors `arxiv_chunks_table` into Lakebase. Delta remains the source of truth. Update the source and synchronize it; do not insert or update application data directly in the managed copy.

Name	Where it is used
`catalog.schema.arxiv_chunks_table`	Delta source in Unity Catalog
`catalog.schema.arxiv_chunks_synced`	Synced table resource in Unity Catalog
`schema.arxiv_chunks_synced`	Table queried through a Postgres connection
`projects/<project>/branches/production`	Target Lakebase branch in the notebook

Creation sequence

1. Validate that the source contains a stable, non-null `id` and populated `embedding` values. Check uniqueness as well as column existence.
2. Enable Delta Change Data Feed for the triggered incremental workflow.
3. Connect to the target Lakebase database and install the extension required by the selected search path.
4. Create the synced table with `id` as its primary key and a vector type override for `embedding`.
5. Wait for the initial copy or the specific update you started to finish, then inspect the destination.

The vector type override matters

A Delta array does not automatically become a native pgvector column. The notebook's creation specification maps `embedding` to `vector(1024)`, matching the configured GTE model. Without the override, an array can arrive as JSONB and vector operators will not behave as intended.

```
# Inside SyncedTableSyncedTableSpec in notebook 2.5:
primary_key_columns=["id"],
type_overrides=[
    SyncedTableSyncedTableSpecTypeOverride(
        column_name="embedding",
        pg_type=(SyncedTableSyncedTableSpecPgSpecificType
                 .PG_SPECIFIC_TYPE_VECTOR),
        size=1024,
    )
],
```

The full notebook sets the triggered policy, branch, database, usage policy, and pipeline storage location. It creates a missing resource or updates an existing one.

Check both project access and database permissions. Owning or accessing a project does not automatically grant every table-level operation. Index creation can require an appropriate database role or an administrator's grant.

Read more Synced tables

Search Lakebase through Postgres

`LakebasePgVectorManager` packages the database connection, query embedding, and retrieval SQL. It obtains a short-lived database credential through the Databricks SDK and uses `psycopg` to connect with TLS. Do not copy a temporary token into source code.

```
from databricks.sdk import WorkspaceClient
from arxiv_curator.vector_search import LakebasePgVectorManager

pgvector = LakebasePgVectorManager(config=cfg, workspace=WorkspaceClient())
results = pgvector.search(
    query="neural networks and deep learning",
    num_results=5,
    year=2026,
)
hybrid = pgvector.hybrid_search(
    query="transformer architecture attention mechanism",
    num_results=5,
    vector_weight=0.7,
)
```

What the search methods do

`search()` embeds the question with `cfg.embedding_endpoint`, excludes null document vectors, applies an optional year predicate, and orders by cosine distance using <=>. It returns `1 - distance` as a similarity score.

`hybrid_search()` combines that vector score with PostgreSQL `ts_rank_cd` over a full-text expression built from title and chunk text. The default weighted sum is 70% vector score and 30% text score. This is the course's illustrative SQL scoring formula, not AI Search's BM25 implementation. The scores have different scales, so tune and evaluate the weighting.

Indexes and the current implementation

pgvector supports native vector index options such as HNSW. The current course code disables the separate HNSW index because the optional Lakebase Search example uses `lakebase_ann` on the same column. `ensure_indexes()` currently creates the GIN index for the full-text expression; notebook 2.5 section 9 creates the ANN index.

A direct cosine query can still execute without an ANN index, but that does not establish acceptable performance on a large table. Likewise, a weighted hybrid query over all eligible rows does not automatically use an ANN index efficiently. Inspect query plans and consider candidate retrieval followed by reranking when scaling beyond the teaching dataset.

Inspect the actual database. Check `pg_typeof(embedding)`, row counts, null vectors, and `pg_indexes`. Confirm the intended index exists after a recreation or schema change. The executable statements are more reliable than an old notebook summary mentioning HNSW.

The lecture recommends this direct Postgres route for the agent. It offers explicit control over metadata predicates and scoring while keeping the source data synchronized from Delta.

The ai_search SQL function

The product name “AI Search” and the SQL function `ai_search()` are different things. The function can search an AI Search index or a Lakebase source. Notebook 2.5 demonstrates both, but the lecture encountered workspace errors on the Lakebase path and does not rely on it for the agent.

Source configurations

Type	Resource and configuration
`vector_search`	AI Search index name, text column, and document identifier column
`lakebase_search`	Lakebase synced table name, text and identifier columns, embedding column, embedding model, and Lakebase project ID

The Lakebase route is a Beta capability. It requires Lakebase Search enabled for the project, `lakebase_vector`, and a `lakebase_ann` index on the vector column. Enabling Lakebase Search is irreversible. The configured embedding endpoint must match the one used to generate the stored vectors.

```
-- Run against the Lakebase database, not Spark SQL:
CREATE EXTENSION IF NOT EXISTS lakebase_vector CASCADE;
CREATE INDEX IF NOT EXISTS arxiv_chunks_synced_embedding_ann
ON arxiv.arxiv_chunks_synced
USING lakebase_ann (embedding vector_cosine_ops);
```

Then call the SQL function from Databricks with the Unity Catalog name of the synced table. Substitute your environment’s names:

```
SELECT ai_search(
  'transformer architecture attention mechanism',
  PARSE_JSON('[{
    "type": "lakebase_search",
    "config": {
      "index_name": "catalog.arxiv.arxiv_chunks_synced",
      "text_col": "text",
      "doc_uri_col": "id",
      "embedding_column": "embedding",
      "embedding_model": "databricks-gte-large-en",
      "lakebase_project_id": "your-project"
    }
  }]'),
  options => map('generate_answer', 'false')
) AS result;
```

The function can do more than nearest-neighbor retrieval, including answer generation by default. The example disables generation so that its purpose is retrieval. Its results and cost are therefore not directly interchangeable with a raw Postgres distance query.

If this Beta route is unavailable, use the direct Postgres workflow described on the previous page. A failure here is not proof that all vector retrieval from Lakebase is unavailable.

Read more ai_search configuration and requirements

Keep the serving data fresh

The lecture refers to Spark Declarative Pipelines on Lakeflow as SDP, previously Delta Live Tables or DLT. Both serving routes use managed synchronization infrastructure. The choice between triggered and continuous processing is a general pipeline operating decision, not a special property of RAG.

Question	Triggered	Continuous
When does it stop?	After processing the available update	Keeps processing source changes
Typical fit	Periodic or explicitly requested refreshes	Low-lag serving requirements
Cost pattern	Compute for update runs	Ongoing processing resources
Course choice	Used for the demonstrated indexes and synced table	Consider only when freshness justifies it

Databricks' general SDP guidance associates triggered processing with updates roughly every ten minutes, hourly, or daily, and continuous processing with shorter freshness targets. These are guidelines, not delivery-time guarantees. Initial loads, source volume, and service-specific support still matter.

Trigger the resource you actually use

```
# AI Search Delta Sync index
index = ai_search.get_index(index_name=managed_name)
index.sync()
index.wait_until_ready(wait_for_updates=True)

# Lakebase Synced Table: the course's Pipelines API approach
name = f"synced_tables/{cfg.catalog}.{cfg.schema}.arxiv_chunks_synced"
status = w.postgres.get_synced_table(name=name).status
update = w.pipelines.start_update(pipeline_id=status.pipeline_id)
# Poll this update_id until completed; fail on FAILED or CANCELED.
```

For Lakebase, notebook 2.5 obtains the pipeline ID from the synced table status. It waits for the particular update it started, rather than treating an already-online table as proof that new data has arrived. The platform also provides Sync now and a Lakeflow Jobs Database Table Sync task.

Operational checks

- Keep the stable primary key and source Change Data Feed intact.
- Track ingestion, synchronization, and serving as separate stages; successful ingestion does not prove the search copy is fresh.
- Check row counts, update state, source timestamps, and the presence of new document IDs.
- Attach the configured usage policy and inspect job failures before starting another update.

The repository defines separate jobs for data processing, AI Search, and synced tables. Their presence does not by itself guarantee an end-to-end schedule. Confirm the actual task dependencies and trigger order in the deployed environment.

Read more SDP pipeline modes

Improve retrieval with evidence

Use a small, repeatable set of realistic questions before changing models or search settings. For each question, identify relevant papers or chunks where possible. Include exact acronyms, broad conceptual questions, date constraints, and questions the collection cannot answer.

Measure the right failure

Metric	What it tells you
Precision@k	How many of the retrieved top-k items are relevant
Recall@k	How many of the known relevant items appear in the top-k
MRR or nDCG	Whether useful items are ranked near the top
Application checks	Answer correctness, citation support, latency, and cost

Ground-truth examples can start with human labels or reviewed synthetic queries. An LLM judge can assist, but inspect its judgments and coverage. A small teaching sample is suitable for learning the workflow; it is not a strong benchmark for a production corpus.

A practical improvement sequence

1. **Try hybrid search.** Compare it with vector-only retrieval on exact terms and semantic questions.
2. **Use metadata filters.** Add known constraints such as publication year rather than asking vector similarity to represent them.
3. **Test reranking.** Judge whether better ordering is worth the additional latency and throughput requirements.
4. **Improve preparation.** Fix parsing mistakes, weak boundaries, and missing source information.
5. **Expand queries.** Try alternate wording, then deduplicate the results.
6. **Improve preparation or rewriting prompts.** Evaluate them independently.
7. **Use adaptive retrieval.** Let the application reformulate a query or decide whether retrieval is needed.
8. **Consider embedding fine-tuning.** Treat it as a later intervention when simpler changes do not address language or domain problems.

Evaluate the final context too

A correct retrieval result can still become a poor prompt. Deduplicate chunks, preserve source identifiers, order evidence clearly, and respect the generation model's budget. If a smaller model compresses the evidence, check that key facts and qualifications survive. Validate the final answer rather than optimizing retrieval scores in isolation.

Authorization is also part of retrieval correctness. Metadata filtering for relevance is not a replacement for enforcing the user's access rights. Keep permission decisions in trusted application and platform controls.

Read more Databricks retrieval quality guide

Practice and implementation map

Build the retrieval foundation on a manageable sample. If your own project does not need every part of this pipeline, use the arXiv dataset for the exercise and carry the relevant patterns back to your application.

Homework

1. **Inspect the processed documents and chunk tables.** Compare at least a few extracted chunks with their PDFs and note any lost structure.
2. **Review chunk sizes and try an alternative.** Explain the trade-off and verify the selected embedding model's input limit.
3. **Create a Lakebase synced table.** Confirm the primary key, vector type, synchronization state, and destination row count.
4. **Run semantic, filtered, and hybrid search.** Reuse the same questions and capture what changes in the returned evidence.
5. **Submit through the course pull-request process.** Include the configuration approach, observations, and remaining limitations.

Where the implementation lives

File	Use it for
`notebooks/2.1_context_engineering_theory.py`	Rewriting, compression, context assembly, and retrieval concepts
`notebooks/2.2_pdf_parsing_ai_parse.py`	Run the data preparation pipeline
`notebooks/2.3_chunking_strategies.py`	Inspect lengths and experiment with boundaries
`notebooks/2.4_databricks_ai_search.py`	Create and query managed and self-managed AI Search indexes
`notebooks/2.5_synced_tables.py`	Sync to Lakebase, query Postgres, and explore ai_search
`src/arxiv_curator/data_processor.py`	Download, parse, extract, clean, and embed
`src/arxiv_curator/vector_search.py`	Database credentials, indexes, and direct retrieval
`project_config.yml`	Environment-specific resources and model endpoint

Next lecture: MCP, tool calling, and agent architecture.